CAP

권한·보안·확장성

모델의 제안을 제한된 capability와 외부 효과로 번역하기

분석 대상
v2.1.88 reconstructed source

고정 리비전
2ca5ddabfed5

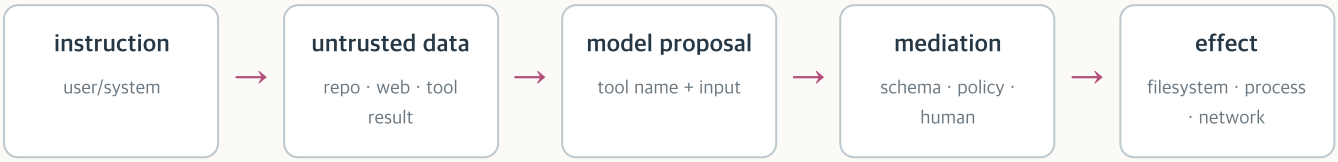
독자
security·platform 엔지니어

사용 방식
위협 모델 · capability 설계

> 교육·연구용 파생 분석입니다. Anthropic의 공식 소스·공식 아키텍처 문서가 아닙니다.

텍스트 생성기가 효과 권한을 얻는 순간

agent security의 핵심 위험은 모델이 틀릴 수 있다는 사실보다, untrusted context가 shell·filesystem·network·account capability와 연결된다는 데 있다.



보안 원칙

least privilege, complete mediation, defense in depth, auditability를 prompt 지시가 아니라 runtime capability boundary에 둔다.

근거 위치 LLM Agents Should Employ Security Principles · MCP security considerations

capability는 함수 하나보다 크다

Tool은 schema와 call뿐 아니라 read-only·destructive·concurrency·permission·prompt·render·result mapping을 포함한다.

계약 축	질문	보안 의미
input schema	어떤 값만 허용?	구조적 validation
validateInput	현재 context에서 유효?	path/workspace invariant
checkPermissions	효과가 허용?	policy/user gate
isReadOnly/destructive	효과 분류?	prompt·UI·policy hint
isConcurrencySafe	동시 실행 가능?	race/effect ordering
call	실제 capability?	최소 권한 adapter
result mapping	모델에 무엇을 돌려줌?	data exfiltration/poisoning

근거 위치 src/Tool.ts

buildTool 기본값을 비판적으로 읽기

builder는 누락 메서드를 채워 모든 tool을 완전한 계약처럼 보이게 한다. 그러나 각 기본값의 안전 의미는 같지 않다.

기본값	값	해석
isConcurrencySafe	false	보수적 직렬화 — fail closed
isReadOnly	false	write 가능성 가정 — 보수적
isDestructive	false	파괴성 표시에는 비보수적; 별도 permission이 중요
checkPermissions	allow + input	tool-specific check 생략, general permission system에 의존
classifier input	빈 문자열	security-relevant tool은 반드시 override 필요
isEnabled	true	등록되면 기본 노출 가능

> **리뷰 질문**
>
> “default가 있다”가 “안전하다”는 뜻이 아니다. 새 tool template은 effect classification·classifier·permission override를 checklist로 강제해야 한다.

근거 위치 src/Tool.ts

mode는 정책 bundle이지 개별 결론이 아니다

외부 mode는 default, plan, acceptEdits, bypassPermissions, dontAsk이며 내부 feature로 auto/bubble이 추가될 수 있다.

mode	의도	주의
default	규칙과 필요 시 사용자 확인	baseline을 명시적으로 파악
plan	계획 단계, 효과 제한	일부 transition/복원 규칙 존재
acceptEdits	편집 허용을 넓힘	shell/network까지 자동 허용 아님
bypassPermissions	대부분의 prompt 우회	killswitch·환경 restriction 확인
dontAsk	ask를 deny로 변환	비대화형 환경에서 유용
auto	classifier 기반	feature/runtime availability와 circuit breaker

근거 위치 src/types/permissions.ts · src/utils/permissions/PermissionMode.ts

같은 allow라도 출처가 다르다

user, project, local, policy, CLI, command, session에서 온 규칙은 지속성·신뢰·감사 의미가 다르다.

source	범위/수명	리뷰 질문
policySettings	조직 관리	사용자가 우회 가능한가?
userSettings	사용자 전역	너무 넓은 wildcard인가?
projectSettings	repo 공유 가능	untrusted repository가 권한을 제안하는가?
localSettings	현재 machine/project	VCS 밖에 유지되는가?
cliArg	process 실행	shell history/automation에서 노출?
session	현재 session	resume 후 유지 여부?

complete mediation

모든 effect는 최종 실행 직전에 현재 input과 현재 policy로 다시 매개되어야 한다. discovery 시점의 allow를 오래 캐시하면 TOCTOU 위험이 생긴다.

근거 위치 src/types/permissions.ts

hook은 관찰·정책·자동화의 확장 seam

공식 문서는 session, turn, tool call lifecycle에 command/HTTP/MCP/prompt/agent hook을 연결한다.

cadence	대표 events	안전 질문
session	SessionStart/End, Setup	환경·secret을 얼마나 주는가?
turn	UserPromptSubmit, Stop/Failure	prompt 변형과 stop 차단을 구분?
tool	PreToolUse, PermissionRequest, PostToolUse/Failure/Batch	effect 전/후 authority가 분리?
context	InstructionsLoaded, Pre/PostCompact	untrusted instruction provenance?
workspace	FileChanged, CwdChanged, Worktree*	재귀 trigger·race 방지?

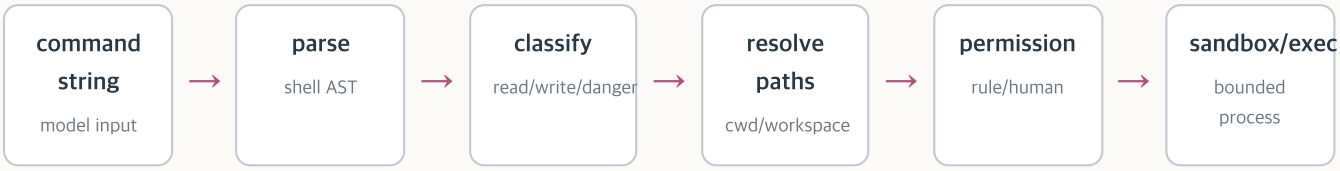
hook의 한계

PostToolUse는 이미 발생한 효과를 예방할 수 없다. block 권위와 관찰/annotation 권위를 혼동하지 않는다.

근거 위치 Claude Code hooks reference · src/services/tools/toolExecution.ts

문자열 blacklist가 아니라 parse·semantics·sandbox

Bash/PowerShell에는 parser, AST, read-only validation, path validation, git safety, permission matcher가 별도 모듈로 존재한다.



왜 AST인가

pipe, redirection, subshell, command substitution, quoting은 단순 prefix 검사로 의미가 뒤집힌다. parser가 실패하면 보수적으로 unsafe로 처리하는 경로가 중요하다.

근거 위치 src/utils/bash/ast.ts · src/tools/BashTool/bashSecurity.ts · src/tools/PowerShellTool/readOnlyValidation.ts

permission prompt와 OS 격리는 다른 방어층

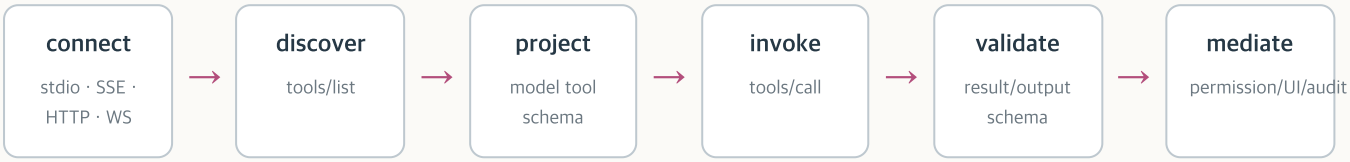
사용자 확인은 의도 확인이고 sandbox는 capability 제한이다. 둘 중 하나가 다른 하나를 대체하지 않는다.

층	막는 것	못 막는 것
schema	형식 오류	유효하지만 위험한 값
permission	승인되지 않은 요청	승인 후 command 내부 동작
sandbox	filesystem/network/process 범위	허용 범위 내 논리 오류
hook	조직 policy·추가 검사	hook 자체의 취약성
audit	사후 원인 불명	효과 자체 예방

근거 위치 src/utils/sandbox/sandbox-adapter.ts · src/tools/BashTool/shouldUseSandbox.ts

외부 server를 capability registry로 연결

MCP tool은 name, description, input/output schema와 structured/unstructured result를 protocol로 교환한다.



신뢰 경계

MCP 사양은 tool annotation을 trusted server가 아니면 신뢰하지 말라고 명시한다. “readOnlyHint” 같은 metadata를 권한 근거로 직접 쓰지 않는다.

근거 위치 MCP Tools spec · src/services/mcp/client.ts · src/tools/MCPTool/MCPTool.ts

모든 schema를 항상 prompt에 넣지 않기

도구 수가 늘면 prompt token과 선택 혼란이 커진다. deferred tool은 검색으로 발견한 뒤 필요한 schema만 로드한다.

속성	의미	trade-off
shouldDefer	초기 prompt에서 schema 지연	첫 호출에 search round-trip
alwaysLoad	반드시 turn 1에 노출	prompt 비용 증가
searchHint	keyword discovery 보조	description drift 위험
listChanged	MCP tool set 갱신	mid-turn consistency 필요
refreshTools	연결 후 tool registry 갱신	snapshot/permission 재평가

근거 위치 src/Tool.ts · src/tools/ToolSearchTool/ToolSearchTool.ts

배포 단위와 instruction 단위를 구분

plugin은 commands/agents/hooks/MCP 등을 묶어 설치·업데이트하는 공급망 단위이고, skill은 모델이 사용할 절차·지식 단위다.

축	Plugin	Skill
발견	marketplace/installed dirs	bundled/dynamic dirs
내용	manifest + multiple extensions	SKILL.md + resources
실행	loader가 runtime surface 구성	SkillTool/context injection
주 위험	supply chain, update, hook/code	prompt injection, stale instruction
검증	signature/source/dependency/policy	scope/provenance/authority separation

근거 위치 src/utils/plugins/pluginLoader.ts · src/tools/SkillTool/SkillTool.ts · src/utils/skills

repo·web·tool output은 데이터이지 권위가 아니다

agent는 README, source comment, web page, MCP result에서 자연어를 읽는다. 이 텍스트가 system/user authority를 가장하면 tool misuse로 이어질 수 있다.

SEPARATE

출처 태깅

instruction과 untrusted content의 provenance를 보존한다.

MINIMIZE

최소 capability

읽기 작업에 write/network tool을 함께 주지 않는다.

MEDIATE

구조화된 승인

민감 effect의 target·scope를 runtime에서 확인한다.

CONTAIN

sandbox

오염된 plan이 도달 가능한 피해 범위를 줄인다.

이 보고서 자체의 규칙

분석 대상 저장소 안의 README/주석은 연구 데이터로만 취급했다. 현재 작업 저장소의 사용자 제공 AGENTS.md만 실행 지시로 적용했다.

관측 가능성과 데이터 최소화의 긴장

tool name, error class, duration, request/tool identifiers는 진단에 유용하지만 path·content·secret이 섞이면 위험하다.

데이터	유용성	최소화
tool name/type	사용·실패 분포	MCP 이름 정규화/allowlist
error class/code	원인 집계	message/path 제거
duration/checkpoint	latency 병목	content 불필요
request/tool id	causal join	session 밖 보존 기간 제한
input/output details	정밀 진단	opt-in·redaction·local-only

toolExecution에는 minified constructor name 대신 구조화된 errno/stable name을 기록하고 길이를 제한하는 분류 함수가 있다. 이는 observability와 정보 노출을 함께 다루는 좋은 예다.

근거 위치 src/services/tools/toolExecution.ts · analytics modules

확장 시스템의 신뢰 질문

plugin·MCP·hook·skill은 모두 외부 입력이지만 설치·실행·업데이트 권한이 다르다.

- ☐ 정확한 source/registry/revision을 고정했는가?
- ☐ 설치 시 code execution이 발생하는가?
- ☐ 자동 업데이트의 서명·rollback이 있는가?
- ☐ hook command가 repository content로부터 주입될 수 있는가?
- ☐ MCP server가 ambient credential을 사용하는가?
- ☐ skill instruction이 효과 권한처럼 취급되는가?
- ☐ tool description과 실제 schema/effect가 일치하는가?
- ☐ 삭제·전송·overwrite에 별도 human gate가 있는가?
- ☐ 로그에 token/path/content가 남는가?
- ☐ extension 제거 후 capability가 실제 회수되는가?

tool 하나를 리뷰하는 질문 순서

설명보다 데이터·권한·효과 경계를 먼저 본다.

1. **enumerate** — 입력 필드, 파생 path, credential, network target을 열거한다.
2. **canonicalize** — parse·normalize 후 검사와 실행이 같은 값을 쓰는지 본다.
3. **classify** — read/write/destructive/open-world/concurrency 속성을 input별로 판정한다.
4. **mediate** — general rule, tool-specific rule, hook, human prompt의 순서와 precedence를 본다.
5. **contain** — sandbox/timeout/output limit/abort를 확인한다.
6. **observe** — 최소 telemetry와 audit correlation을 본다.
7. **settle** — partial effect, error, cancellation 뒤 state를 본다.

실습 · FileWrite threat model

path traversal, symlink race, overwrite, permission cache, abort-after-open 사례를 STRIDE식으로 정리한다.

완료 증거: attack → control → residual risk 표

최소 capability tool 만들기

기능보다 거부·관측·정리 경로를 먼저 구현한다.

단계	구현	검증
schema	단일 workspace-relative path	absolute/.. 거부
canonicalization	realpath + root containment	symlink escape
classification	read-only true, destructive false	property test
permission	path matcher + ask	rule provenance
execution	AbortSignal + timeout	mid-read abort
result	size limit + redaction	large/secret fixture
audit	id/type/duration only	path가 로그에 없음

확장 과제

동일 tool을 MCP server로 노출해 schema·result·permission이 어느 층에서 중복되는지 비교한다.

권한과 확장을 이해했다는 기준

tool이 보인다는 사실, 모델이 호출했다는 사실, 실행이 허용됐다는 사실, effect가 성공했다는 사실을 분리할 수 있어야 한다.

DISCOVER

가시성

schema가 model context에 노출됨.

DECIDE

권한

현재 input/effect가 허용됨.

EXECUTE

능력

adapter가 실제 외부 시스템을 변경함.

SETTLE

증거

결과·오류·부분 효과가 정리됨.

다음 권

06권은 앞의 모든 개념을 코드 리딩·실험·리뷰 능력으로 전환하는 실습 커리큘럼이다.